**D597 Task 1:** Relational Database Design and Implementation

## Part 1: Design

### A. Select one of the provided scenarios to complete the following:

I chose Scenario 2 for Task 1, which focuses on EcoMart, an emerging online marketplace for sustainable products. I designed a structured relational database built for flexibility and scalable growth.

***1. Describe a business problem that can be solved with a database solution and is in alignment with the chosen scenario.***

As EcoMart continues to grow its product catalog and user base, the lack of a centralized, scalable system to manage its dynamic data will pose challenges in the future. EcoMart has expressed the desire to expand its database to maintain product descriptions, pricing, availability, sustainability certifications, and user reviews. However, relying on a flat CSV file will make it increasingly difficult to integrate, update, and maintain this data. With a structured relational database EcoMart will better maintain data consistency, have access to real-time analytics, and empower stakeholders to make informed decisions to promote business growth and eco-conscious practices.

***2. Propose a data structure to solve the identified business problem.***

A structured relational database model is ideal for EcoMart as an emerging online marketplace. This model utilizes primary and foreign keys to organize data into related tables to promote data integrity, reduce redundancy, and enable flexibility. Each core entity, such as region, country, item type, sales channel, and sales history, can be normalized into discrete tables to promote data consistency and ease of maintenance as the business grows.

Unlike the CSV-based solution EcoMart has implemented, relation databases can efficiently handle large and growing datasets. Excel is limited to one million rows and lacks structural enforcement. Features like parallel processing, indexing, and SQL queries make relational databases far more efficient

for handling large datasets and performing complex analyses. Proxify (2024) noted that SQL outpaces Excel in both speed and scalability when working with substantial volumes of data.

Relational databases also offer built-in tools such as access control, audit logging, and automated backups. These are essential for securing sensitive customer and business information. This setup supports EcoMart's current operational needs and lays the groundwork for future expansion.

***3. Justify why a database solution will solve the identified business problem.***

Currently, EcoMart maintains its data in a structured comma-separated values (CSV) file in the First Normal Form (1NF); this provides a basic level of organization. However, the CSV already has 100,000 records and is expected to grow significantly as the business scales. The limitations of a flat-file system will become evident. Increased record volume will lead to longer query times, higher risk of redundancy, and greater difficulty in maintaining data integrity.

Migrating the data to a structured relational database in the Third Normal Form (3NF) will address these issues by organizing the data into logically separated tables. This approach will reduce redundant data, enforce data consistency across the system, and improve querying performance through indexing and performance-tuning methods. Leveraging SQL to work with the data offers more efficient runtimes than processing large CSVs in Excel or similar tools.

In addition to operational benefits, this transition supports EcoMart's eco-conscious mission. Well-optimized SQL queries on indexed databases consume less energy than processing large spreadsheet files. This aligns the database efficiency with the company's broader sustainability goals.

***4. Explain how the business data will be used within the database solution.***

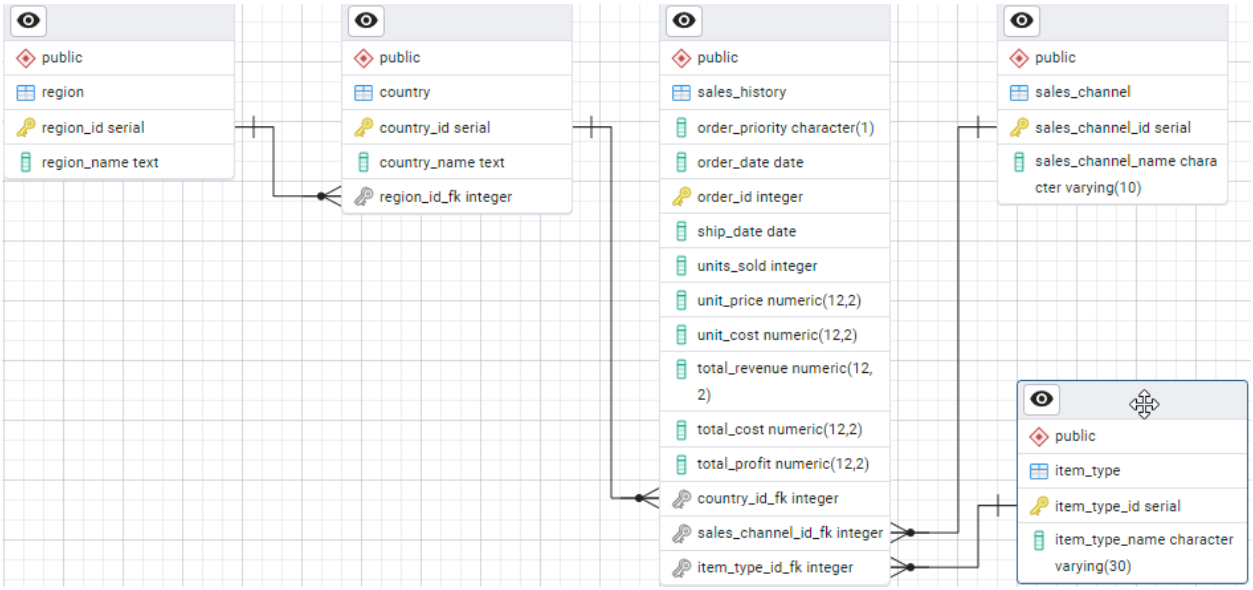
Based on EcoMart's current dataset, the business data will be organized into structured tables – region, country, item type, sales channel, and sales history. These tables will be connected through defined relationships that reflect real-world business linkages, such as which products are sold in which regions.

This data will support various business activities once implemented as a relational database using pgAdmin. For example, EcoMart can run queries to identify high-performing sales regions, underperforming item types, and emerging trends in order fulfillment.

The database will store EcoMart's data and support its reporting and analytical needs by being the foundation for business intelligence and decision-making, aiding EcoMart in aligning its business growth strategies with its eco-conscious mission

**B. Create a logical data model for storing data in the database solution.**

The logical data model for EcoMart's relational database consists of five core entities: region, country, item_type, sales_channel, and sales_history. Each entity is represented as a separate table with clearly defined primary keys and foreign key relationships that mirror real-world business relations. The model is normalized to the Third Normal Form (3NF) to eliminate redundancy and ensure data consistency.**The logical data model for EcoMart's relational database consists of five core entities: region, country, item_type, sales_channel, and sales_history. Each entity is represented as a separate table with clearly defined primary keys and foreign key relationships that mirror real-world business relations. The model is normalized to the Third Normal Form (3NF) to eliminate redundancy and ensure data consistency.**




**C. Describe the database objects and storage, identifying the file attributes within the database solution.**

1. Region

- Attributes:
  - region_id (Primary Key, serial)
  - region_name (text)
- Each region represents a geographic classification used to group countries.

2. Country

- Attributes:
  - country_id (Primary Key, serial)
  - country_name (text)
  - region_id_fk (Foreign Key → region.region_id)

- Each country is associated with one region.

3. Item Type

- Attributes:
  - item_type_id (Primary Key, serial)
  - item_type_name (varchar(30))
- Represents categories of products sold by EcoMart.

4. Sales Channel

- Attributes:
  - sales_channel_id (Primary Key, serial)
  - sales_channel_name (varchar(10))
- Represents the medium through which a product is sold (e.g., online, retail).

5. Sales History

- Attributes:
  - order_id (Primary Key, integer)
  - order_priority (char(1))
  - order_date (date)
  - ship_date (date)
  - units_sold (integer)
  - unit_price, unit_cost, total_revenue, total_cost, total_profit (numeric(12,2))
  - country_id_fk (Foreign Key → country.country_id)
  - item_type_id_fk (Foreign Key → item_type.item_type_id)
  - sales_channel_id_fk (Foreign Key → sales_channel.sales_channel_id)
- This table acts as a fact table, capturing transactional sales data and connecting it to dimension tables (country, item type, sales channel).

**D. Discuss how the proposed database design addresses scalability concerns, including strategies that align with the chosen scenario.**

The relational database design is more scalable than EcoMart's current CSV-based data management system. While CSVs may appear convenient for quick updates, they do not offer the structure, data validation, or relational links needed to grow and manage the system effectively over time. As EcoMart grows, the relational model will support the seamless integration of new data without disrupting the existing structure.

With normalized tables, it is easy to grow the database; for instance, new item types can be inserted into the item_type table without modifying the sales_history structure. Similarly, adding new regions will only require an insert into the region table and linking new countries accordingly. The 3NF design will minimize data redundancy and ensure referential integrity across expanding datasets.

The relational model facilitates performance scalability. Indexing foreign keys and frequently queried fields will ensure efficient joins and filtering even as the volume of sales transactions increases. The separation of fact and dimension tables supports analytical flexibility and enables database administrators to partition or archive historical data when needed.

**E. Outline the privacy and security measures that should be implemented in the proposed database design.**

Since EcoMart deals with a large amount of sensitive customer and transaction data, keeping that information secure is critical. Without proper protection, they could face data breaches, legal issues, hefty fines, and lasting damage to their reputation.

All production-facing data must be encrypted at rest and in transit to ensure data privacy and integrity. Role-based access should be implemented to ensure that users only have access to specific data they are authorized to view or modify.

EcoMart should enforce strong authentication mechanisms, including multi-factor authentication (MFA) for employees and contractors. MFA options may include hardware tokens (e.g., RSA) or time-based one-time passcodes issued by the InfoSec team for time-limited access. Access control policies should follow the principle of least privilege and be reviewed regularly, ideally quarterly, to ensure that only authorized personnel retain access.

Audit logging must be enabled to capture database access events, data modifications, and administrative actions. The audit logs should be stored securely with limited access and monitored to detect unauthorized or suspicious activity.

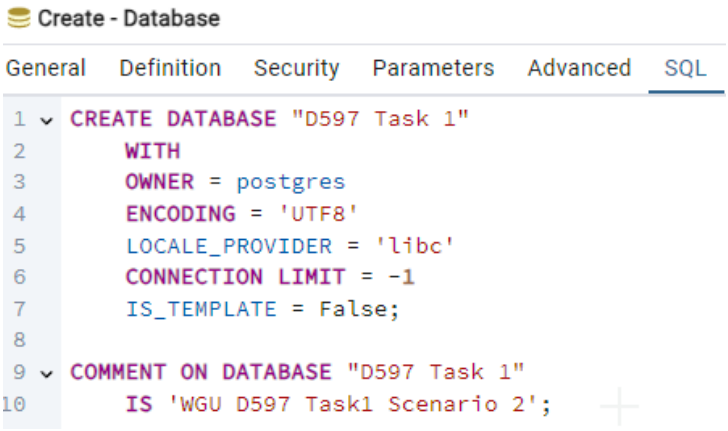
EcoMart should conduct periodic security assessments, including internal penetration tests, vulnerability scans, and third-party security audits. Running disaster recovery drills and maintaining up-to-date backup procedures will ensure EcoMart is prepared to respond effectively to data loss or compromise scenarios.

## Part 2: Implementation

**F. Implement the proposed database design in the WGU Virtual Lab environment by completing the following:**

***1. Write script to create a database instance named "D597 Task 1" using the appropriate query language, based on the logical data model in part B. Provide a screenshot showing the script and the database instance in the platform.***

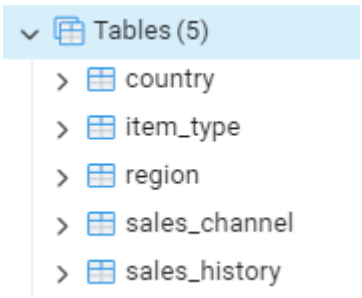
The following script created the database instance 'D597 Task 1'. This database will serve as the foundation for implementing the logical data model described in Part B.



Screenshot of the scripts used to create the initial database tables. This structure was intentionally denormalized for the initial import process. It will be normalized later in the database implementation by replacing name fields with foreign key references to the appropriate dimension tables.

```
--create region table
CREATE TABLE region (
    region_id SERIAL PRIMARY KEY,
    region_name TEXT
);
--create country table
CREATE TABLE country (
    country_id SERIAL PRIMARY KEY,
    country_name TEXT,
    region_id_fk INTEGER,
    region_name TEXT
);
-- create item_type table
CREATE TABLE item_type (
    item_type_id SERIAL PRIMARY KEY,
    item_type_name VARCHAR(30)
);
--create sales_channel table
CREATE TABLE sales_channel (
    sales_channel_id SERIAL PRIMARY KEY,
    sales_channel_name VARCHAR(10)
);
--create initial sales_history table. includes all cols for data import.
CREATE TABLE sales_history (
    region_name TEXT,
    country_name TEXT,
    item_type_name VARCHAR(30),
    sales_channel_name VARCHAR(10),
    order_priority CHAR(1),
    order_date DATE,
    order_id INTEGER PRIMARY KEY,
    ship_date DATE,
    units_sold INTEGER,
    unit_price NUMERIC(12,2),
    unit_cost NUMERIC(12,2),
    total_revenue NUMERIC(12,2),
    total_cost NUMERIC(12,2),
    total_profit NUMERIC(12,2));
```

Screenshot showing the database and tables in the Object Explorer for pgAdmin:



***2. Write script to import the data records from the chosen scenario CSV files into the database instance. Provide a screenshot showing the script and the data correctly inserted or mapped into the database.***

Provide a screenshot showing the script and the data correctly inserted or mapped into the database.

The PostgreSQL script to import EcoMart's CSV file into the sales_history table

```
46  copy public.sales_history (region_name,
47  country_name, item_type_name, sales_channel_name,
48  order_priority, order_date, order_id, ship_date, units_sold,
49  unit_price, unit_cost, total_revenue, total_cost, total_profit)
50  FROM 'C:/Users/PaceW/Desktop/WGU/Courses/D597-D~1/SCENAR~2/SCENAR~1/100000~1.CSV' DELIMITER ',' CSV HEADER ESCAPE ''''
```

Data Output | Messages | Notifications

```
COPY 100000

Query returned successfully in 369 msec.
```

Confirmed that all 100,000 rows were loaded into the database.

```
7   SELECT COUNT(region_name) FROM sales_history;
```

Data Output | Messages | Notifications

| | count bigint |
|---|---|
| 1 | 100000 |

Scripts to SELECT data from the sales_history table and INSERT it into the appropriate table.

```
54  DELETE FROM region;
55  INSERT INTO region (region_name)
56  SELECT DISTINCT region_name
57  FROM sales_history;
58
59  DELETE FROM country;
60  INSERT INTO country(country_name, region_name)
61  SELECT DISTINCT country_name, region_name
62  FROM sales_history;
63
64  DELETE FROM item_type;
65  INSERT INTO item_type(item_type_name)
66  SELECT DISTINCT item_type_name
67  FROM sales_history;
68
69  DELETE FROM sales_channel;
70  INSERT INTO sales_channel(sales_channel_name)
71  SELECT DISTINCT sales_channel_name
72  FROM sales_history;
```

Verifying the results in all tables.

Region

```
73  SELECT *
74  FROM region
```

Data Output | Messages | Notifications

| | region_id [PK] integer | region_name text |
|---|---|---|
| 1 | 1 | Australia and Oceania |
| 2 | 2 | Asia |
| 3 | 3 | Central America and the Caribbean |
| 4 | 4 | North America |
| 5 | 5 | Sub-Saharan Africa |
| 6 | 6 | Middle East and North Africa |
| 7 | 7 | Europe |

Country

```
74  SELECT *
75  FROM country;
```

Data Output | Messages | Notifications

| | country_id [PK] integer | country_name text | region_id_fk integer | region_name text |
|---|---|---|---|---|
| 1 | 186 | Greenland | [null] | North America |
| 2 | 187 | Nauru | [null] | Australia and Oceania |
| 3 | 188 | South Africa | [null] | Sub-Saharan Africa |
| 4 | 189 | Italy | [null] | Europe |
| 5 | 190 | Seychelles | [null] | Sub-Saharan Africa |

item_type

```
73  SELECT *
74  FROM item_type
```

Data Output | Messages | Notifications

| | item_type_id [PK] integer | item_type_name character varying (30) |
|---|---|---|
| 1 | 1 | Fruits |
| 2 | 2 | Baby Food |
| 3 | 3 | Beverages |
| 4 | 4 | Vegetables |
| 5 | 5 | Clothes |

Sales_channel

```
76  ALTER TABLE sales_history
77      ADD country_id_fk INT,
78      ADD sales_channel_id_fk INT,
79      ADD item_type_id_fk INT;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 31 msec.
```

Adding the necessary columns to the sales_history table to support foreign keys.


```
76  ALTER TABLE sales_history
77      ADD country_id_fk INT,
78      ADD sales_channel_id_fk INT,
79      ADD item_type_id_fk INT;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 31 msec.
```


Populating the Foreign Keys across all tables.

```
82  UPDATE country c
83  SET region_id_fk = r.region_id
84  FROM region r
85  WHERE c.region_name = r.region_name;
86
87  UPDATE sales_history sh
88  SET country_id_fk = c.country_id
89  FROM country c
90  WHERE sh.country_name = c.country_name;
91
92  UPDATE sales_history sh
93  SET sales_channel_id_fk = sc.sales_channel_id
94  FROM sales_channel sc
95  WHERE sh.sales_channel_name = sc.sales_channel_name;
96
97  UPDATE sales_history sh
98  SET item_type_id_fk = it.item_type_id
99  FROM item_type it
100 WHERE sh.item_type_name = it.item_type_name;
```

Data Output | Messages | Notifications

```
UPDATE 100000

Query returned successfully in 1 secs 257 msec.
```

Dropping the columns from the sales_history table now that they have become redundant within the database.

```
112 ALTER TABLE sales_history
113     DROP COLUMN region_name,
114     DROP COLUMN country_name,
115     DROP COLUMN item_type_name,
116     DROP COLUMN sales_channel_name;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 61 msec.
```

```
116 ALTER TABLE country
117     DROP COLUMN region_name;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 36 msec.
```

Adding Foreign Key Constraints across the tables.

```
118 ALTER TABLE country
119     ADD CONSTRAINT fk_region FOREIGN KEY (region_id_fk) REFERENCES region(region_id);
120 ALTER TABLE sales_history
121     ADD CONSTRAINT fk_country FOREIGN KEY (country_id_fk) REFERENCES country(country_id),
122     ADD CONSTRAINT fk_item_type FOREIGN KEY (item_type_id_fk) REFERENCES item_type(item_type_id),
123     ADD CONSTRAINT fk_sales_channel FOREIGN KEY (sales_channel_id_fk) REFERENCES sales_channel(sales_channel_id);
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 63 msec.
```

***3. Write script for three queries to retrieve specific information from the database that will help to solve the identified business problem. Provide a screenshot showing the script for each query and each query successfully executed.***

1) Query 1: Regional Profitability. As an emerging marketplace, EcoMart needs visibility into its regional performance. For some region(s), EcoMart may want to increase the marketing budget, run sales promotions, reduce investment, or withdraw from the region.

```
SELECT r.region_name AS region,
  TO_CHAR(SUM(sh.total_profit), 'FM$999,999,999,999.00') AS total_region_profit,
  TO_CHAR(SUM(sh.units_sold), 'FM999,999,999,999') AS units_sold,
  TO_CHAR(100.0 * SUM(sh.total_profit) / SUM(SUM(sh.total_profit)) OVER (), 'FM999.00') || '%' AS percent_of_total
FROM sales_history sh
JOIN country c ON sh.country_id_fk = c.country_id
JOIN region r ON c.region_id_fk = r.region_id
GROUP BY r.region_name
ORDER BY SUM(sh.total_profit) DESC;
```

| | region<br>text | total_region_profit<br>text | units_sold<br>text | percent_of_total<br>text |
|---|---|---|---|---|
| 1 | Sub-Saharan Africa | $10,306,312,642.23 | 129,890,681 | 26.15% |
| 2 | Europe | $10,080,579,491.05 | 128,664,731 | 25.58% |
| 3 | Asia | $5,707,511,516.76 | 72,822,259 | 14.48% |
| 4 | Middle East and North Africa | $4,979,534,378.88 | 63,251,625 | 12.64% |
| 5 | Central America and the Caribbean | $4,287,210,522.47 | 53,871,798 | 10.88% |
| 6 | Australia and Oceania | $3,175,423,561.38 | 40,797,618 | 8.06% |
| 7 | North America | $872,551,616.84 | 10,845,905 | 2.21% |

Total rows: 7 Query complete 00:00:00.094

2) Query 2: Item Type Sales Performance. This query allows EcoMart to identify underperforming item types that may not justify continued production, storage, or promotion. This information can be used to make decisions to reduce operational waste and environmental impact.

```
SELECT it.item_type_name,
    TO_CHAR(SUM(sh.units_sold), 'FM999,999,999,999') AS units_sold,
    TO_CHAR(SUM(sh.total_profit), 'FM$999,999,999,999.00') AS total_profit_item_type,
    TO_CHAR(100.0 * SUM(sh.total_profit) / SUM(SUM(sh.total_profit)) OVER (), 'FM999.00') || '%' AS percent_of_total_profit
FROM sales_history sh
JOIN item_type it
ON sh.item_type_id_fk = it.item_type_id
GROUP BY it.item_type_name
ORDER BY SUM(total_profit) DESC, SUM(sh.units_sold) DESC;
```

| | item_type_name character varying (30) | units_sold text | total_profit_item_type text | percent_of_total_profit text |
|---|---|---|---|---|
| 1 | Cosmetics | 41,924,464 | $7,289,406,555.68 | 18.50% |
| 2 | Household | 41,458,795 | $6,870,966,095.35 | 17.43% |
| 3 | Office Supplies | 42,293,330 | $5,339,532,912.50 | 13.55% |
| 4 | Baby Food | 41,911,620 | $4,017,647,893.20 | 10.19% |
| 5 | Cereal | 42,254,418 | $3,743,318,890.62 | 9.50% |
| 6 | Clothes | 41,773,440 | $3,067,841,433.60 | 7.78% |
| 7 | Vegetables | 41,254,836 | $2,604,417,796.68 | 6.61% |
| 8 | Meat | 41,745,367 | $2,387,834,992.40 | 6.06% |
| 9 | Snacks | 41,699,092 | $2,299,287,932.88 | 5.83% |
| 10 | Personal Care | 41,517,766 | $1,040,435,215.96 | 2.64% |
| 11 | Beverages | 41,514,213 | $650,112,575.58 | 1.65% |
| 12 | Fruits | 40,797,276 | $98,321,435.16 | .25% |

Total rows: 12 | Query complete 00:00:00.087

3) Query 3: Fulfillment performance. This query helps EcoMart spot potential issues in their order fulfillment by showing the average time between when an order is placed and when it ships. Catching delays early gives the company a chance to fix problems before they affect customer satisfaction.

```
SELECT it.item_type_name,
    ROUND(AVG(sh.ship_date - sh.order_date), 2) AS avg_days_to_ship
FROM sales_history sh
JOIN item_type it
ON sh.item_type_id_fk = it.item_type_id
GROUP BY it.item_type_name
ORDER BY avg_days_to_ship DESC;
```

| | item_type_name character varying (30) | avg_days_to_ship numeric |
|---|---|---|
| 1 | Office Supplies | 25.23 |
| 2 | Household | 25.19 |
| 3 | Vegetables | 25.10 |
| 4 | Clothes | 25.08 |
| 5 | Baby Food | 25.07 |
| 6 | Cosmetics | 25.06 |
| 7 | Meat | 25.03 |
| 8 | Snacks | 25.02 |
| 9 | Beverages | 24.97 |
| 10 | Fruits | 24.97 |
| 11 | Personal Care | 24.95 |
| 12 | Cereal | 24.76 |

Total rows: 12 Query complete 00:00:00.099

***4. Apply optimization techniques to improve the run time of your queries from part F3, providing output results via a screenshot.***

As was described in part A.3, the database was normalized into third standard form (3NF) to eliminate transitive dependencies and reduce redundancy. This design separates the sales_history fact table from its supporting dimension tables (region, country, item_type, and sales_channel). These changes improve data integrity and enhance querying performance.

To demonstrate this, I executed a query using the RANK() window function to identify the top 10 countries by total profit. On the optimized 3NF database, the query was executed in 49 milliseconds, as shown in the screenshot below.

For the comparison, I executed a similar query on a non-normalized database version that retained redundant data fields within the main table. The query took 59 milliseconds, as reflected in the screenshot below.

This comparison validates that restricting the schema for normalization significantly improves performance for analytical queries by reducing data duplication and optimizing join paths.

**Optimized database results:**

Here, I queried to find the top 10 countries by profit utilizing the RANK() windows function, which took 49 milliseconds.

```
SELECT c.country_name,
TO_CHAR(SUM(sh.total_profit), 'FM$999,999,999,999') AS sum_of_profits,
RANK() OVER(ORDER BY SUM(total_profit) DESC) AS rank_by_profit
FROM sales_history sh
JOIN country c
ON sh.country_id_fk = c.country_id
GROUP BY c.country_name
LIMIT 10;
```

Data Output | Messages | Notifications

| | country_name text | sum_of_profits text | rank_by_profit bigint |
|---|---|---|---|
| 1 | Sudan | $249,523,549 | 1 |
| 2 | Hungary | $241,860,844 | 2 |
| 3 | Federated States of Micronesia | $241,045,260 | 3 |
| 4 | Liberia | $239,114,337 | 4 |
| 5 | Australia | $238,598,456 | 5 |
| 6 | Benin | $238,294,600 | 6 |
| 7 | New Zealand | $237,880,636 | 7 |
| 8 | Bahrain | $237,638,193 | 8 |
| 9 | Marshall Islands | $236,922,446 | 9 |
| 10 | Malta | $234,376,856 | 10 |

Total rows: 10 Query complete 00:00:00.049

**Nonoptimized database results:**

When executing a similar query against the unoptimized database version, we see an increased run time of 59 milliseconds.

```
SELECT country_name,
TO_CHAR(SUM(total_profit), 'FM$999,999,999,999') AS sum_of_profits,
RANK() OVER(ORDER BY SUM(total_profit) DESC) AS rank_by_profit
FROM sales_history
GROUP BY country_name
LIMIT 10;
```

Data Output | Messages | Notifications

| | country_name text | sum_of_profits text | rank_by_profit bigint |
|---|---|---|---|
| 1 | Sudan | $249,523,549 | 1 |
| 2 | Hungary | $241,860,844 | 2 |
| 3 | Federated States of Micronesia | $241,045,260 | 3 |
| 4 | Liberia | $239,114,337 | 4 |
| 5 | Australia | $238,598,456 | 5 |
| 6 | Benin | $238,294,600 | 6 |
| 7 | New Zealand | $237,880,636 | 7 |
| 8 | Bahrain | $237,638,193 | 8 |
| 9 | Marshall Islands | $236,922,446 | 9 |
| 10 | Malta | $234,376,856 | 10 |

Total rows: 10 Query complete 00:00:00.059

Another optimization technique that can be implemented is indexing. Specifically for the item_type_id, sales_channel_id, order_date and ship_date fields. As EcoMarts data volume grows, these indexes will improve join speeds and filter performance during analytical queries.

```sql
--optimization scripts to create indexes.
--improves JOIN performance with the Item_Type table.
CREATE INDEX idx_item_type_id_fk ON Sales_History(Item_Type_ID_FK);

--improves JOIN performance with the Sales_Channel table.
CREATE INDEX idx_sales_channel_id_fk ON Sales_History(Sales_Channel_ID_FK);

--speeds up queries that filter or sort by Order_Date (e.g., for trend analysis or fulfillment lag).
CREATE INDEX idx_order_date ON Sales_History(Order_Date);

--index on Ship_Date if you often analyze shipping timelines.
CREATE INDEX idx_ship_date ON Sales_History(Ship_Date);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 212 msec.
```

Total rows: | Query complete 00:00:00.212

**Below I have confirmed indexing improved the run times of the three queries from F3:**

Query 1: Regional Profitability.

Before

```sql
--three queries for business problems.
--Q1: regional profitability.
SELECT r.region_name AS region,
    TO_CHAR(SUM(sh.total_profit), 'FM999,999,999,999.00') AS total_region_profit,
    TO_CHAR(SUM(sh.units_sold), 'FM999,999,999') AS units_sold,
    TO_CHAR(100.0 * SUM(sh.total_profit) / SUM(SUM(sh.total_profit)) OVER (),'FM999.00') || '%' AS percent_of_total
FROM sales_history sh
JOIN country c ON sh.country_id_fk = c.country_id
JOIN region r ON c.region_id_fk = r.region_id
GROUP BY r.region_name
ORDER BY SUM(sh.total_profit) DESC;
```

Data Output | Messages | Notifications

| | region text | total_region_profit text | units_sold text | percent_of_total text |
|---|---|---|---|---|
| 1 | Sub-Saharan Africa | 10,306,312,642.23 | 129,890,681 | 26.15% |
| 2 | Europe | 10,080,579,491.05 | 128,664,731 | 25.58% |
| 3 | Asia | 5,707,511,516.76 | 72,822,259 | 14.48% |
| 4 | Middle East and North Africa | 4,979,534,378.88 | 63,251,625 | 12.64% |
| 5 | Central America and the Caribbean | 4,287,210,522.47 | 53,871,798 | 10.88% |
| 6 | Australia and Oceania | 3,175,423,561.38 | 40,797,618 | 8.06% |
| 7 | North America | 872,551,616.84 | 10,845,905 | 2.21% |

Total rows: 7 | Query complete 00:00:00.190

After

```sql
--Q1: regional profitability.
SELECT r.region_name AS region,
    TO_CHAR(SUM(sh.total_profit), 'FM999,999,999,999.00') AS total_region_profit,
    TO_CHAR(SUM(sh.units_sold), 'FM999,999,999') AS units_sold,
    TO_CHAR(100.0 * SUM(sh.total_profit) / SUM(SUM(sh.total_profit)) OVER (),'FM999.00') || '%' AS percent_of_total
FROM sales_history sh
JOIN country c ON sh.country_id_fk = c.country_id
JOIN region r ON c.region_id_fk = r.region_id
GROUP BY r.region_name
```

Data Output | Messages | Notifications

| | region text | total_region_profit text | units_sold text | percent_of_total text |
|---|---|---|---|---|
| 1 | Sub-Saharan Africa | 10,306,312,642.23 | 129,890,681 | 26.15% |
| 2 | Europe | 10,080,579,491.05 | 128,664,731 | 25.58% |
| 3 | Asia | 5,707,511,516.76 | 72,822,259 | 14.48% |
| 4 | Middle East and North Africa | 4,979,534,378.88 | 63,251,625 | 12.64% |
| 5 | Central America and the Caribbean | 4,287,210,522.47 | 53,871,798 | 10.88% |
| 6 | Australia and Oceania | 3,175,423,561.38 | 40,797,618 | 8.06% |
| 7 | North America | 872,551,616.84 | 10,845,905 | 2.21% |

Total rows: 7 | Query complete 00:00:00.088

Query 2: Item Type Sales Performance.

Before

```sql
--Q2: item type sales performance.
SELECT it.item_type_name,
    TO_CHAR(SUM(sh.units_sold), 'FM999,999,999') AS units_sold,
    TO_CHAR(SUM(sh.total_profit), 'FM999,999,999,999.00') AS total_profit_item_type,
    TO_CHAR(100.0 * SUM(sh.total_profit) / SUM(SUM(sh.total_profit)) OVER (),'FM999.00') || '%' AS percent_of_total_profit
FROM sales_history sh
JOIN item_type it ON sh.item_type_id_fk = it.item_type_id
GROUP BY it.item_type_name
ORDER BY SUM(sh.total_profit) DESC, SUM(sh.units_sold) DESC;
```

Data Output | Messages | Notifications

| | item_type_name character varying (30) | units_sold text | total_profit_item_type text | percent_of_total_profit text |
|---|---|---|---|---|
| 1 | Cosmetics | 41,924,464 | 7,289,406,555.68 | 18.50% |
| 2 | Household | 41,458,795 | 6,870,966,095.35 | 17.43% |
| 3 | Office Supplies | 42,293,330 | 5,339,532,912.50 | 13.55% |
| 4 | Baby Food | 41,911,620 | 4,017,647,893.20 | 10.19% |
| 5 | Cereal | 42,254,418 | 3,743,318,890.62 | 9.50% |
| 6 | Clothes | 41,773,440 | 3,067,841,433.60 | 7.78% |
| 7 | Vegetables | 41,254,836 | 2,604,417,796.68 | 6.61% |
| 8 | Meat | 41,745,367 | 2,387,834,992.40 | 6.06% |
| 9 | Snacks | 41,699,092 | 2,299,287,932.88 | 5.83% |
| 10 | Personal Care | 41,517,766 | 1,040,435,215.96 | 2.64% |
| 11 | Beverages | 41,514,213 | 650,112,575.58 | 1.65% |
| 12 | Fruits | 40,797,276 | 98,321,435.16 | .25% |

Total rows: 12 | Query complete 00:00:00.117

After

```sql
--Q2: item type sales performance.
SELECT it.item_type_name,
    TO_CHAR(SUM(sh.units_sold), 'FM999,999,999') AS units_sold,
    TO_CHAR(SUM(sh.total_profit), 'FM999,999,999,999.00') AS total_profit_item_type,
    TO_CHAR(100.0 * SUM(sh.total_profit) / SUM(SUM(sh.total_profit)) OVER (),'FM999.00') || '%' AS percent_of_total_profit
FROM sales_history sh
JOIN item_type it ON sh.item_type_id_fk = it.item_type_id
GROUP BY it.item_type_name
ORDER BY SUM(sh.total_profit) DESC, SUM(sh.units_sold) DESC;
```

Data Output | Messages | Notifications

| | item_type_name character varying (30) | units_sold text | total_profit_item_type text | percent_of_total_profit text |
|---|---|---|---|---|
| 1 | Cosmetics | 41,924,464 | 7,289,406,555.68 | 18.50% |
| 2 | Household | 41,458,795 | 6,870,966,095.35 | 17.43% |
| 3 | Office Supplies | 42,293,330 | 5,339,532,912.50 | 13.55% |
| 4 | Baby Food | 41,911,620 | 4,017,647,893.20 | 10.19% |
| 5 | Cereal | 42,254,418 | 3,743,318,890.62 | 9.50% |
| 6 | Clothes | 41,773,440 | 3,067,841,433.60 | 7.78% |
| 7 | Vegetables | 41,254,836 | 2,604,417,796.68 | 6.61% |
| 8 | Meat | 41,745,367 | 2,387,834,992.40 | 6.06% |
| 9 | Snacks | 41,699,092 | 2,299,287,932.88 | 5.83% |
| 10 | Personal Care | 41,517,766 | 1,040,435,215.96 | 2.64% |
| 11 | Beverages | 41,514,213 | 650,112,575.58 | 1.65% |
| 12 | Fruits | 40,797,276 | 98,321,435.16 | .25% |

Total rows: 12 | Query complete 00:00:00.088

Query 3: Fulfillment performance.

Before

```
70  --Q3: fullfillment performance.
71  SELECT it.item_type_name,
72      ROUND(AVG(sh.ship_date - sh.order_date), 2) AS avg_days_to_ship
73  FROM sales_history sh
74  JOIN item_type it ON sh.item_type_id_fk = it.item_type_id
75  GROUP BY it.item_type_name
76  ORDER BY avg_days_to_ship DESC;
```

Data Output | Messages | Notifications

| | item_type_name character varying (30) | avg_days_to_ship numeric |
|---|---|---|
| 1 | Office Supplies | 25.23 |
| 2 | Household | 25.19 |
| 3 | Vegetables | 25.10 |
| 4 | Clothes | 25.08 |
| 5 | Baby Food | 25.07 |
| 6 | Cosmetics | 25.06 |
| 7 | Meat | 25.03 |
| 8 | Snacks | 25.02 |
| 9 | Beverages | 24.97 |
| 10 | Fruits | 24.97 |
| 11 | Personal Care | 24.95 |
| 12 | Cereal | 24.76 |

Total rows: 12 | Query complete 00:00:00.101

After

```sql
--Q3: fullfillment performance.
SELECT it.item_type_name,
    ROUND(AVG(sh.ship_date - sh.order_date), 2) AS avg_days_to_ship
FROM sales_history sh
JOIN item_type it ON sh.item_type_id_fk = it.item_type_id
GROUP BY it.item_type_name
ORDER BY avg_days_to_ship DESC;
```

Data Output | Messages | Notifications

| | item_type_name character varying (30) | avg_days_to_ship numeric |
|---|---|---|
| 1 | Office Supplies | 25.23 |
| 2 | Household | 25.19 |
| 3 | Vegetables | 25.10 |
| 4 | Clothes | 25.08 |
| 5 | Baby Food | 25.07 |
| 6 | Cosmetics | 25.06 |
| 7 | Meat | 25.03 |
| 8 | Snacks | 25.02 |
| 9 | Beverages | 24.97 |
| 10 | Fruits | 24.97 |
| 11 | Personal Care | 24.95 |
| 12 | Cereal | 24.76 |

Total rows: 12 | Query complete 00:00:00.093

References

Team, P. C. (2024, October 17). SQL vs Excel: Choosing the right tool for your data needs. proxify. https://proxify.io/articles/sql-vs-excel